

Flask Framework: Building Web App With Python



Shubham Sarda



May 7, 2023



23 min read

☰ Table of contents

Introduction

Basic steps to use Flask

Blueprint

Rendering HTML Template

Template Variables

Parameter

> URL Parameter

Show more ▼

Introduction

Flask is a popular web framework for building web applications in Python. It is designed to be simple, lightweight, and easy to use, with a modular architecture that allows developers to choose the components they need and add or remove

functionality as required.

One of the key benefits of Flask is its simplicity and ease of use. It has a small code base and minimal dependencies, which makes it easy to learn and use for beginners, while still providing the flexibility and power needed for more complex applications. Additionally, Flask is highly customizable, allowing developers to build web applications that fit their specific needs and requirements.

Overall, Flask is a popular choice for building web applications in Python due to its simplicity, flexibility, and powerful features.

Basic steps to use Flask

Flask is a web framework for Python that allows you to build web applications. Here are the basic steps to use Flask:

1. Install Flask: You can install Flask using pip, the Python package manager. Open your terminal and enter the following command:

```
pip install flask
```

1. Create a new Flask application: Create a new Python file and import the Flask module. Then create an instance of the Flask class:

```
from flask import Flask  
  
app = Flask(__name__)
```

1. Define a route: In Flask, a route is a URL that a user can access. You can define a route using the `@app.route` decorator:

```
@app.route('/')  
  
def home():  
    return 'Hello, World!'
```

In this example, the `home` function is called when a user visits the root URL `/`.

1. Run the Flask application: To run the Flask application, add the following code to the bottom of your Python file:

```
if __name__ == '__main__':  
    app.run(debug=True)
```

This code runs the Flask application in debug mode. You can start the Flask application by running the Python file from the terminal:

```
python app.py
```

1. Test the Flask application: Open your web browser and go to `http://localhost:5000/`. You should see the message "Hello, World!".

This is a very basic example of using Flask. Flask has many features for building web applications, such as handling form submissions, managing user authentication, and connecting to databases. You can refer to Flask's documentation for more information on how to use these features.

Blueprint

A Blueprint is a way to organize and modularize a Flask application by grouping related routes, view functions, templates, and static files. Blueprints are especially helpful when building larger applications, as they provide a cleaner and more maintainable structure. Blueprints are registered with the main Flask application, and their routes become part of the overall application.

Here's a simple example demonstrating how to create and register a Blueprint:

1. Create a new directory named `my_blueprint` for your Blueprint module.
2. Inside the `my_blueprint` directory, create a file named `__init__.py`. This file will define and export the Blueprint:

`my_blueprint/__init__.py`:

```
from flask import Blueprint

my_blueprint = Blueprint('my_blueprint', __name__)

from . import views
```

In this file, a new Blueprint named 'my_blueprint' is created, and the views module is imported, ensuring that the routes are registered with the Blueprint.

1. Inside the `my_blueprint` directory, create a file named `views.py` to define the Blueprint's routes and view functions:

`my_blueprint/views.py` :

```
from . import my_blueprint

@my_blueprint.route('/hello')
def hello():
    return 'Hello from Blueprint!'
```

Here, a route is defined for the URL path `"/hello"` within the `my_blueprint` Blueprint.

1. In your main application, import and register the Blueprint:

`app.py` :

```
from flask import Flask
from my_blueprint import my_blueprint

app = Flask(__name__)
app.register_blueprint(my_blueprint)

if __name__ == '__main__':
    app.run()
```

In `app.py`, the `my_blueprint` Blueprint is imported from the `my_blueprint` module and registered with the main Flask application using the `register_blueprint` method.

Now, when you run the `app.py` file, it will create a Flask application, register the Blueprint, and start the development server. When a user visits the URL path

"/hello", the `hello` function defined in the `my_blueprint/views.py` file will be executed, and the text "Hello from Blueprint!" will be displayed in their browser.

You can create and register multiple Blueprints to better organize your application and separate functionalities.

Rendering HTML Template

To render an HTML template with Flask, you can use the `render_template` function provided by Flask's `flask` module. Here are the steps:

1. Create an HTML template: Create an HTML file with the template you want to use. For example, let's create a template called `home.html` :

```
<!DOCTYPE html>
<html>
  <head>
    <title>Home</title>
  </head>
  <body>
    <h1>Welcome to my website!</h1>
  </body>
</html>
```

1. Import the `render_template` function: Import the `render_template` function from the `flask` module:

```
from flask import Flask, render_template

app = Flask(__name__)
```

1. Define a route: Define a route that will render the template. In this example, we will render the `home.html` template when the user visits the root URL `/`:

```
@app.route('/')
def home():
    return render_template('home.html')
```

1. Run the Flask application: Run the Flask application using the `app.run()` method:

```
if __name__ == '__main__':
    app.run()
```

1. Test the Flask application: Open your web browser and go to <http://localhost:5000/>. You should see the message "Welcome to my website!" rendered in the `home.html` template.

Note: The `render_template` function will look for templates in a directory called `templates` in the same directory as your Flask application. Make sure you create a `templates` directory and place your HTML templates in it.

This is a simple example of how to render an HTML template with Flask. Flask's `render_template` function also allows you to pass variables to your templates, include other templates, and more. Check out Flask's documentation for more information on how to use `render_template`.

Template Variables

In Flask, you can pass variables to your HTML templates using the

`render_template` function. Here are the steps:

1. Create an HTML template: Create an HTML file with the template you want to use. For example, let's create a template called `home.html` :

```
<!DOCTYPE html>
<html>
  <head>
    <title>{{ title }}</title>
  </head>
  <body>
    <h1>Welcome to {{ site_name }}!</h1>
    <p>{{ message }}</p>
  </body>
</html>
```

Note that we have used the Jinja template engine syntax to include template variables inside the HTML code.

1. Import the `render_template` function: Import the `render_template` function from the `flask` module:

```
from flask import Flask, render_template

app = Flask(__name__)
```

1. Define a route: Define a route that will render the template. In this example, we

will render the `home.html` template when the user visits the root URL `/`:

```
@app.route('/')
def home():
    title = 'Home'
    site_name = 'My Website'
    message = 'Welcome to my website!'
    return render_template('home.html', title=title, site_name=site_name,
```

Here, we have defined three variables `title`, `site_name`, and `message`, and passed them to the `render_template` function as keyword arguments.

1. Run the Flask application: Run the Flask application using the `app.run()` method:

```
if __name__ == '__main__':
    app.run()
```

1. Test the Flask application: Open your web browser and go to <http://localhost:5000/>. You should see the message "Welcome to My Website!" rendered in the `home.html` template.

Note: In the template, we have used the variables `title`, `site_name`, and `message` that were passed to the `render_template` function.

This is a simple example of how to use template variables in Flask. You can pass any number of variables to your templates and use them in your HTML code using Jinja syntax. Check out Flask's documentation for more information on how to use Jinja templates.

Parameter

URL Parameter

In Flask, you can use URL parameters to dynamically generate content based on user input. Here are the steps:

1. Define a route with a parameter: Define a route that includes a parameter in the URL. For example, let's define a route that takes a parameter called `name`:

```
@app.route('/hello/<name>')
def hello(name):
    return f'Hello, {name}!'
```

In this example, we have defined a route that takes a parameter called `name` using the `<name>` syntax. The function that handles this route is called `hello` and takes the `name` parameter as an argument.

1. Run the Flask application: Run the Flask application using the `app.run()` method:

```
if __name__ == '__main__':
    app.run()
```

1. Test the Flask application: Open your web browser and go to <http://localhost:5000/hello/John>. You should see the message "Hello, John!" displayed in your web browser.

In this example, we have passed the parameter `John` to the `hello` function using the URL `/hello/John`. The `hello` function then uses this parameter to dynamically generate the message "Hello, John!".

Note: You can define routes with multiple parameters and different parameter types such as strings, integers, and floats. In addition, you can use optional parameters and specify default values for parameters. Check out Flask's documentation for more information on how to use URL parameters in Flask.

Query Parameter

In Flask, you can use query parameters to pass additional data to your routes. Query parameters are added to the end of a URL using the `?` symbol, and multiple parameters are separated by the `&` symbol. Here are the steps:

1. Define a route with query parameters: Define a route that includes query parameters. For example, let's define a route that takes two query parameters called `name` and `age`:

```
@app.route('/hello')
def hello():
    name = request.args.get('name')
    age = request.args.get('age')
    return f'Hello, {name}! You are {age} years old.'
```

In this example, we have defined a route called `/hello` that uses the `request.args` object to access the query parameters `name` and `age`. The `request.args.get()` method is used to get the value of a query parameter.

1. Run the Flask application: Run the Flask application using the `app.run()` method:

```
if __name__ == '__main__':  
    app.run()
```

1. Test the Flask application: Open your web browser and go to <http://localhost:5000/hello?name=John&age=30>. You should see the message "Hello, John! You are 30 years old." displayed in your web browser.

In this example, we have passed two query parameters `name` and `age` to the `hello` function using the URL `/hello?name=John&age=30`. The `hello` function then uses these parameters to dynamically generate the message "Hello, John! You are 30 years old."

Note: You can use query parameters to pass any type of data to your routes, such as strings, integers, and booleans. In addition, you can set default values for query parameters and use conditional logic to handle missing or invalid parameters. Check out Flask's documentation for more information on how to use query parameters in Flask.

Conditional Rendering

You can use Jinja2's control structures, such as the `{% if %}` statement, to add conditional rendering in your HTML templates. Here's an example:

Assume you want to show a welcome message to the user only if they're logged in. You can pass a variable named `logged_in` from your Flask view to the template,

and then use the `{% if %}` statement to conditionally render the welcome message.

1. Update your Flask view to pass the `logged_in` variable to the template:

```
# views.py
@main.route('/')
def index() -> ResponseReturnValue:
    logged_in = True # This value should come from your authentication logic
    return render_template('main/index.html', logged_in=logged_in)
```

1. Update your template to use the `{% if %}` statement for conditional rendering:

```
<!-- templates/main/index.html -->
{% extends "main/base.html" %}

{% block content %}
    <h1>Welcome to My Website</h1>
    {% if logged_in %}
        <p>Welcome back, user! We're glad to see you again.</p>
    {% endif %}
{% endblock %}
```

In this example, the `{% if logged_in %}` statement checks if the `logged_in` variable is `True`. If it is, the welcome message will be rendered. If not, the message will be skipped. Note that the `{% endif %}` tag is used to close the `{% if %}` statement.

You can also use `{% else %}` and `{% elif %}` within an `{% if %}` statement

for more complex conditional rendering:

```
{% if user_role == 'admin' %}
    <p>Welcome, admin! You have full access to the website.</p>
{% elif user_role == 'moderator' %}
    <p>Welcome, moderator! You have limited access to the website.</p>
{% else %}
    <p>Welcome, user! You have basic access to the website.</p>
{% endif %}
```

In this example, different welcome messages are rendered based on the value of the `user_role` variable.

Return JSON Response

In Flask, you can return JSON data from a route using the `jsonify` method provided by the Flask library. Here are the steps:

1. Import the `jsonify` method: Import the `jsonify` method from the Flask library:

```
from flask import jsonify
```

1. Define a route that returns JSON: Define a route that returns JSON data. For example, let's define a route that returns a dictionary containing a person's name and age:

```
@app.route('/person')
```

```
def person():  
    data = {'name': 'John', 'age': 30}  
    return jsonify(data)
```

In this example, we have defined a route called `/person` that returns a dictionary containing a person's name and age. The `jsonify` method is used to convert the dictionary to a JSON response.

1. Run the Flask application: Run the Flask application using the `app.run()` method:

```
if __name__ == '__main__':  
    app.run()
```

1. Test the Flask application: Open your web browser and go to <http://localhost:5000/person>. You should see the JSON data displayed in your web browser:

```
{  
  "name": "John",  
  "age": 30  
}
```

In this example, the `person` function returns the JSON data `{ "name": "John", "age": 30 }`.

Note: You can return any type of JSON-serializable data from a route, such as lists, tuples, and nested dictionaries. Check out Flask's documentation for more information on how to return JSON data in Flask.

Handle Errors (404)

To handle 404 (Not Found) errors in Flask, you can create a custom error handler using the `app.errorhandler()` decorator. This allows you to return a custom response, such as rendering an error template, when a 404 error occurs.

Follow these steps to create a custom 404 error handler:

1. Create a new template for the 404 error page, for example, `404.html` :

```
<!-- templates/404.html -->
<!DOCTYPE html>
<html>
  <head>
    <title>Page Not Found</title>
  </head>
  <body>
    <h1>Error 404: Page Not Found</h1>
    <p>We're sorry, but the page you're looking for cannot be found.</p>
  </body>
</html>
```

1. In your `views.py` file, create a custom error handler function using the `app.errorhandler()` decorator, and render the 404 template:

```
# views.py
from flask import render_template

# ... your other imports, routes, and functions ...
```

```
@main.app_errorhandler(404)
def page_not_found(e):
    return render_template('404.html'), 404
```

In this example, the `@ main.app _errorhandler(404)` decorator indicates that the `page_not_found()` function should be used as the custom error handler for 404 errors. When a 404 error occurs, the `page_not_found()` function is called, and the custom 404 template is rendered with a 404 status code.

Remember to update your `app.py` to import the custom error handler:

```
# app.py
from flask import Flask
from views import main

app = Flask(__name__)
app.register_blueprint(main)

if __name__ == '__main__':
    app.run(debug=True)
```

With this custom error handler in place, your Flask application will display the custom 404 error page whenever a user tries to access a page that doesn't exist.

Redirect

In Flask, you can redirect a user to another page using the `redirect()` function.

Here are the steps:

1. Import the `redirect()` function: Import the `redirect()` function from the Flask library:

```
from flask import redirect
```

1. Define a route that redirects to another page: Define a route that redirects to another page using the `redirect()` function. For example, let's define a route that redirects the user from the home page (`/`) to the about page (`/about`):

```
@app.route('/')
def home():
    return redirect('/about')
```

In this example, we have defined a route called `/` (the home page) that redirects the user to the `/about` page.

1. Run the Flask application: Run the Flask application using the `app.run()` method:

```
if __name__ == '__main__':
    app.run()
```

1. Test the Flask application: Navigate to `http://localhost:5000/` in your web browser. You should be redirected to the `/about` page.

In this example, the `home` function redirects the user to the `/about` page using the `redirect()` function.

Note: You can also pass a URL or route name to the `redirect()` function to redirect the user to a different page. Check out Flask's documentation for more information on how to use the `redirect()` function in Flask.

Include Static File In Template

To add a static image to a Flask application, you can follow these steps:

1. Create a `static` directory: Create a directory called `static` in your Flask application directory. This is where you will store your static files, including images.
2. Store the image in the `static` directory: Save the image file in the `static` directory you created in step 1. For example, let's say you have an image file called `my_image.jpg`. You would save this file in the `static` directory so that the file path is `static/my_image.jpg`.
3. Link to the image in your HTML templates: In your HTML templates, you can link to the image file using the `img` tag and the `src` attribute. For example, let's say you want to display the `my_image.jpg` file in an HTML template. You would use the following code:

```
 ResponseReturnValue:
    projects_list = [
        {'name': 'Project 1', 'description': 'This is the first project.'}
        {'name': 'Project 2', 'description': 'This is the second project.'}
        {'name': 'Project 3', 'description': 'This is the third project.'}
    ]
    return render_template('main/projects.html', projects=projects_list)
```

1. Update your template to use the `{% for %}` loop to iterate over the `projects` list and render the project details:

```
<!-- templates/main/projects.html -->
{% extends "main/base.html" %}

{% block content %}
    <h1>Projects</h1>
    <ul>
        {% for project in projects %}
            <li>
                <h2>{{ project.name }}</h2>
                <p>{{ project.description }}</p>
            </li>
        {% endfor %}
    </ul>
{% endblock %}
```

In this example, the `{% for project in projects %}` statement iterates over the `projects` list, and for each item (a dictionary containing the project's name and description), it renders a list item with the project's details. The `{% endfor %}` tag is used to close the `{% for %}` loop.

You can also use the `else` clause within a `for` loop to render content when the

iterable is empty:

```
{% for project in projects %}
    <li>
        <h2>{{ project.name }}</h2>
        <p>{{ project.description }}</p>
    </li>
{% else %}
    <li>No projects to display.</li>
{% endfor %}
```

In this case, if the `projects` list is empty, the content inside the `{% else %}` clause will be rendered.

Base Template

To create a base template in Flask, you can follow these steps:

1. Create a `templates` directory: Create a directory called `templates` in your Flask application directory. This is where you will store your HTML templates.
2. Create a base template: Create a base HTML template that will be used as the foundation for all of your other HTML templates. The base template should include the common elements that will appear on every page of your application, such as the header, footer, and navigation menu.

For example, you could create a file called `base.html` in the `templates` directory with the following contents:

```

<!DOCTYPE html>

<html>

<head>
    <title>{% block title %}{% endblock %}</title>
</head>
<body>
    <header>
        <h1>My Flask Application</h1>
        <nav>
            <ul>
                <li><a href="{{ url_for('home') }}">Home</a></li>
                <li><a href="{{ url_for('about') }}">About</a></li>
                <li><a href="{{ url_for('contact') }}">Contact</a></li>
            </ul>
        </nav>
    </header>
    <main>
        {% block content %}{% endblock %}
    </main>
    <footer>
        <p>&copy; 2023 My Flask Application</p>
    </footer>
</body>
</html>

```

In this example, we have created a base template that includes a header with a navigation menu, a main section for the content of each page, and a footer with a copyright notice. The `{% block %}` tags are used to define sections of the template that will be overridden in child templates.

1. Create child templates: Create child templates that will extend the base template and override the `{% block %}` tags with their own content. For example, you could create a file called `home.html` in the `templates` directory with the following contents:

```
{% extends 'base.html' %}

{% block title %}Home - My Flask Application{% endblock %}

{% block content %}
    <h2>Welcome to My Flask Application</h2>
    <p>This is the home page of my Flask application.</p>
{% endblock %}
```

In this example, we have extended the `base.html` template using the `{% extends %}` tag, overridden the `{% block title %}` tag with the title for the home page, and overridden the `{% block content %}` tag with the content for the home page.

1. Render the child templates: Render the child templates using the `render_template()` function in your Flask application routes. For example, you could create a route for the home page using the following code:

```
@app.route('/')
def home():
    return render_template('home.html')
```

In this example, we are using the `render_template()` function to render the `home.html` template and return the resulting HTML to the client.

By following these steps, you can create a base template in Flask and use it to generate consistent HTML across all of the pages in your application.

Portfolio Template

Creating a simple HTML and CSS multi-page portfolio template for a Flask project involves creating several files: HTML templates, a CSS file for styling, and a Flask app file. Below is a basic example of how you can set up these files:

1. Create a new folder for your Flask project:

```
/portfolio_flask
  /static
    /css
      - style.css
  /templates
    - base.html
    - index.html
    - about.html
    - projects.html
    - contact.html
  - app.py
```

1. Create the base.html file in the /templates folder. This file will contain the basic structure of your website and will be extended by other templates.

```
<!-- base.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
  <title>{% block title %}Portfolio{% endblock %}</title>
</head>
<body>
```

```

<header>
  <nav>
    <ul>
      <li><a href="{{ url_for('index') }}">Home</a></li>
      <li><a href="{{ url_for('about') }}">About</a></li>
      <li><a href="{{ url_for('projects') }}">Projects</a></li>
      <li><a href="{{ url_for('contact') }}">Contact</a></li>
    </ul>
  </nav>
</header>

<main>
  {% block content %}{% endblock %}
</main>

</body>
</html>

```

1. Create the index.html, about.html, projects.html, and contact.html files in the /templates folder. These files will extend the base.html file.

```

<!-- index.html -->
{% extends "base.html" %}

{% block title %}Home{% endblock %}

{% block content %}
  <h1>Welcome to my portfolio</h1>
  <p>This is the home page.</p>
{% endblock %}

```

```

<!-- about.html -->

```

```
{% extends "base.html" %}

{% block title %}About{% endblock %}

{% block content %}
    <h1>About me</h1>
    <p>This is the about page.</p>
{% endblock %}
```

```
<!-- projects.html -->
{% extends "base.html" %}

{% block title %}Projects{% endblock %}

{% block content %}
    <h1>My projects</h1>
    <p>This is the projects page.</p>
{% endblock %}
```

```
<!-- contact.html -->
{% extends "base.html" %}

{% block title %}Contact{% endblock %}

{% block content %}
    <h1>Contact me</h1>
    <p>This is the contact page.</p>
{% endblock %}
```

1. Create the style.css file in the /static/css folder. This file will contain the styles for your website.

```
/* style.css */

body {
  font-family: Arial, sans-serif;
  line-height: 1.6;
}

header {
  background: #333;
  padding: 1rem 0;
}

nav ul {
  padding: 0;
  list-style: none;
}

nav ul li {
  display: inline;
  margin-right: 1rem;
}

nav ul li a {
  color: #fff;
  text-decoration: none;
}

nav ul li a:hover {
  color: #ccc;
}

h1, h2, h3
```

Common Doubts

What does `if __name__ == '__main__':` do?

In Python, the `if __name__ == '__main__':` block is used to ensure that the code inside it is only executed when the script is run directly as a program, and not when it is imported as a module in another script.

When Python runs a script, it sets a special variable called `__name__` to `"__main__"` for that script. This allows you to differentiate between a script that is being run directly and a script that is being imported as a module in another script.

For example, if you have a file called `my_script.py` and you run it from the command line using `python my_script.py`, the `__name__` variable will be set to `"__main__"`. However, if you import `my_script` in another file using `import my_script`, the `__name__` variable will be set to `"my_script"`.

Here's an example of how the `if __name__ == '__main__':` block might be used in a Flask application:

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return 'Hello, world!'

if __name__ == '__main__':
    app.run(debug=True)
```

In this example, the Flask app is defined and a route is added to it. The `if __name__ == '__main__':` block at the end of the file checks if the script is being

run directly (i.e., if `__name__` is `"__main__"`) and, if so, it starts the Flask development server by calling the `app.run()` method.

By including the `if __name__ == '__main__':` block in your Python code, you can ensure that certain code is only executed when the script is run directly, and not when it is imported as a module in another script.

What does Flask(`__name__`) mean?

In Flask, `Flask(__name__)` creates a new instance of the Flask class. The `__name__` argument is used to determine the name of the application's module or package.

When you create a new Flask instance, you are creating a new application object that will handle incoming HTTP requests and route them to the appropriate view function. The Flask instance is also used to configure the application by setting various properties such as the template folder, static folder, and more.

Here's an example:

```
from flask import Flask

app = Flask(__name__)
```

In this example, we are creating a new Flask application instance called `app`. The `__name__` argument tells Flask that the name of the application's module is the same as the name of the variable (`app`) that we are assigning the Flask instance to.

By default, Flask assumes that your application is located in a single module, so it uses the `__name__` argument to find resources such as templates and static files

relative to the module. If you are using a package structure for your application, you can specify the name of the package instead of the module by passing it to the Flask constructor instead of `__name__`.

What is Jinja?

Jinja is a templating engine used in web development and is a popular choice for building templates in Flask web applications. Jinja allows developers to define templates using a combination of static HTML and dynamic template variables.

Jinja templates are essentially text files that contain a mixture of HTML, expressions, and control statements. The expressions and control statements are evaluated by the Jinja templating engine, and replaced with their corresponding values at runtime.

For example, here is a simple Jinja template that displays a list of names:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Names List</title>
  </head>
  <body>
    <ul>
      {% for name in names %}
        <li>{{ name }}</li>
      {% endfor %}
    </ul>
  </body>
</html>
```

In this template, the `{% for %}` and `{% endfor %}` statements define a loop

that iterates over a list of names, and the `{{ name }}` expression outputs the value of each name in the list.

Jinja provides a rich set of control structures, including conditionals, loops, and filters, which can be used to build complex templates. It also supports template inheritance, which allows you to define a base template with common layout and structure, and then extend it in child templates with additional content.

Overall, Jinja is a powerful and flexible templating engine that provides a convenient way to build dynamic, data-driven web pages in Flask.

[🔖 Flask](#)[🔖 Python](#)[🔖 Web Development](#)[🔖 Programming](#)[🔖 Framework](#)

Shubham Sarda

Senior Software Engineer & Instructor



Shubham Sarda is a full-stack developer, coding mentor, and tech educator with years of industry experience. Passionate about making coding simple, he has helped thousands of students master Python, Django, JavaScript, React and full-stack development through hands-on learning.

His teaching approach focuses on breaking down complex concepts into easy-to-follow lessons while ensuring students gain real-world coding skills. Whether you're a beginner or an experienced developer looking to upskill, Shubham's courses provide structured guidance, industry insights, and practical projects.

At Unwired Learning, he's here to help you learn faster, code smarter, and build confidently. 🚀



Empowering developers through quality education



Quick Links

[About Us](#)

[Contact](#)

[Testimonials](#)

[Blog](#)

Popular Courses

[Python Masterclass](#)

[JavaScript Masterclass](#)

[React Masterclass](#)

[Data Structures & Algorithms](#)

Others

[Verify Certificate](#)

[Terms of Service](#)

[Privacy Policy](#)

[FAQ](#)